

AArch64 Assembly Programming: Arithmetic, Logic, & Control Flow

ALU mechanics, shifted-register operands, bitwise operations,
condition flags, structured branches, and conditional selection

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

Topic 4: Core Lecture Outline

① ALU Operations: Arithmetic, Shifting, & Bitwise Logic

- Addition, subtraction, multiplication, division, and shifted-register operands.
- Bitwise logic (`and`, `orr`, `eor`, `bic`) and bitfield extraction (`ubfx`).

② Condition Flags & Comparison Mechanics

- NZCV flag generation, comparison instructions (`cmp`, `tst`), and signed vs. unsigned branches.

③ Control Flow: Branches & Branchless Selection

- Translating loops and `if-else` structures; branch-and-test (`cbz/cbnz`).
- Branch penalties and branchless conditional selection (`csetl`, `cset`).

PART 1

ALU Operations: Arithmetic, Shifting, & Bitwise Logic

Arithmetic Primitives: Addition & Subtraction

- Many AArch64 data-processing instructions use a **3-operand non-destructive format**:

op destination, source1, source2

Instruction	Operation	Flags Updated?
add x0, x1, x2	$x_0 \leftarrow x_1 + x_2$	No
adds x0, x1, x2	$x_0 \leftarrow x_1 + x_2$	Yes (Updates NZCV)
sub x0, x1, x2	$x_0 \leftarrow x_1 - x_2$	No
subs x0, x1, x2	$x_0 \leftarrow x_1 - x_2$	Yes (Updates NZCV)
neg x0, x1	$x_0 \leftarrow 0 - x_1$	No (Alias for sub x0, xzr, x1)

Immediate Constant Arithmetic

add x0, x1, #42 adds a 12-bit immediate (0 to 4095). 32-bit registers (w0–w30) follow identical syntax.

Shifted-Register Operands and Shift Operations

- The AArch64 ISA allows many instructions to encode a shifted register operand directly within the instruction word.
- Execution latency for shifted operands depends on the specific processor implementation.

Shift Type	Mnemonic	Operation Meaning
Logical Shift Left	<code>lsl #k</code>	Equivalent to multiplication by 2^k modulo the register width (zero-fill).
Logical Shift Right	<code>lsr #k</code>	Equivalent to floor division by 2^k (zero-fill).
Arithmetic Shift Right	<code>asr #k</code>	Sign-preserving right shift (replicates sign bit; differs from truncation).

Integrated Shift-Add Examples

<code>add x0, x1, x2, lsl #3</code>	$\implies x_0 \leftarrow x_1 + (x_2 \times 8)$	(Array indexing by 8-byte doubleword)
<code>sub x0, x1, x2, lsr #1</code>	$\implies x_0 \leftarrow x_1 - (x_2/2)$	(Subtract half of unsigned value)

Integer Multiplication & Division

- AArch64 defines instructions for integer multiplication and division, leaving the underlying microarchitectural execution units implementation-dependent.

Instruction	RTL Semantic Action	Description
<code>mul x0, x1, x2</code>	$x_0 \leftarrow x_1 \times x_2$	64-bit integer product (lower 64 bits)
<code>madd x0, x1, x2, x3</code>	$x_0 \leftarrow x_3 + (x_1 \times x_2)$	Multiply-Add
<code>msub x0, x1, x2, x3</code>	$x_0 \leftarrow x_3 - (x_1 \times x_2)$	Multiply-Subtract
<code>sdiv x0, x1, x2</code>	$x_0 \leftarrow x_1 / x_2$	Signed division
<code>udiv x0, x1, x2</code>	$x_0 \leftarrow x_1 / x_2$	Unsigned division

Division by Zero

AArch64 division by zero does **not** raise an exception; it returns 0 in the destination register.

Bitwise Logic Primitives

- Bitwise operations apply independently to each corresponding bit position.

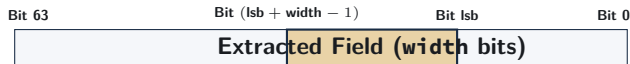
Instruction	Operation	Bitwise RTL Formula
<code>and x0, x1, x2</code>	Bitwise AND	$x_0 \leftarrow x_1 \wedge x_2$
<code>orr x0, x1, x2</code>	Bitwise OR	$x_0 \leftarrow x_1 \vee x_2$
<code>eor x0, x1, x2</code>	Bitwise XOR	$x_0 \leftarrow x_1 \oplus x_2$
<code>bic x0, x1, x2</code>	Bit Clear (AND NOT)	$x_0 \leftarrow x_1 \wedge (\sim x_2)$
<code>mvn x0, x1</code>	Bitwise NOT	$x_0 \leftarrow \sim x_1$ (Alias: <code>orn x0, xzr, x1</code>)

Bit Clear (`bic`) Idiom

`bic x0, x1, x2` sets bits to 0 in x_1 wherever the mask in x_2 has 1s.

Bitfield Extraction: `ubfx`

- AArch64 provides bitfield instructions that extract packed fields without explicit manual mask-and-shift steps.
- `ubfx xd, xn, #lsb, #width`: Unsigned bitfield extract.



Example: Extract 8-bit Field Starting at Bit 16

Standard Approach:	<code>lsr x0, x1, #16</code>	$\rightarrow$	<code>and x0, x0, #0xff</code>	(2 instructions)
Bitfield Extract:	<code>ubfx x0, x1, #16, #8</code>			1 instruction!

PART 2

Condition Flags & Comparison Mechanics

The NZCV Condition Flags

- Flag bits reside in pstate and record status of the most recent instruction that updates NZCV.
- Common flag-setting forms include instructions with an “s” suffix and explicit comparisons.

Flag	Name	Condition Set to 1
N	Negative	The most-significant bit of the result is 1.
Z	Zero	Result is exactly zero.
C	Carry	Unsigned carry-out (addition) or no borrow (subtraction).
V	Overflow	The signed mathematical result cannot be represented in the destination width.

Carry Flag in Subtraction

In AArch64 subtraction ($A - B$), $C = 1$ indicates $A \geq B$ (no borrow required). $C = 0$ indicates $A < B$ (unsigned borrow occurred).

Comparison Instructions: `cmp` and `tst`

- Comparisons evaluate values, set NZCV flags, and discard the computed output.

Instruction	Equivalent Machine Operation	Flags Set
<code>cmp x0, x1</code>	<code>subs xzr, x0, x1</code>	Sets NZCV based on $x_0 - x_1$
<code>cmn x0, x1</code>	<code>adds xzr, x0, x1</code>	Sets NZCV based on $x_0 + x_1$ (Compare Negative)
<code>tst x0, x1</code>	<code>ands xzr, x0, x1</code>	Sets N and Z from result; clears C and V to 0 (Bit)

Checking Flags After `cmp x0, x1`

- If $x_0 == x_1$: **Z flag is set to 1.**
- If $x_0 < x_1$ (signed): **N $\neq$ V.**
- If $x_0 < x_1$ (unsigned): **C = 0.**

Condition Codes: Signed vs. Unsigned Branches

Branch	Meaning	Flag Condition	Comparison Context
b.eq	Equal	$Z == 1$	Signed / Unsigned
b.ne	Not Equal	$Z == 0$	Signed / Unsigned
b.lt	Signed Less Than ($<$)	$N \neq V$	Signed
b.le	Signed Less or Equal ($\leq$)	$(Z == 1) \vee (N \neq V)$	Signed
b.gt	Signed Greater Than ($>$)	$(Z == 0) \wedge (N == V)$	Signed
b.ge	Signed Greater or Equal ($\geq$)	$N == V$	Signed
b.lo	Unsigned Lower ($<$)	$C == 0$	Unsigned / Addresses
b.hs	Unsigned Higher or Same ($\geq$)	$C == 1$	Unsigned / Addresses
b.hi	Unsigned Higher ($>$)	$(C == 1) \wedge (Z == 0)$	Unsigned / Addresses
b.ls	Unsigned Lower or Equal ($\leq$)	$(C == 0) \vee (Z == 1)$	Unsigned / Addresses

Usage Context

Unsigned conditions are appropriate when the program semantics treat the values as unsigned or address-like quantities.

Control Flow: Branches & Selection

Fused Branch-and-Test: **cbz** and **cbnz**

- **Compare and Branch on Zero (cbz) / Non-Zero (cbnz):**
 - Tests register contents directly; does **not** alter NZCV flags.
 - Saves 1 instruction compared to an explicit `cmp + b.eq`.

Null Pointer Validation

```
// If ptr == NULL, jump  
cbz x0, .Lhandle_null  
// Safe memory access  
ldr x1, [x0]
```

Loop Counter Decrement

```
.Lloop:  
sub x1, x1, #1    // count- (flags not needed)  
cbnz x1, .Lloop   // Loop until count == 0
```

Single Bit Check: `tbz x0, #0, label` branches if bit 0 is zero (even number test).

Translating if-then-else Blocks

High-Level C Code

```
if (x > y) {  
    result = x - y;  
} else {  
    result = y - x;  
}
```

AArch64 Assembly Mapping

```
// Assume: x, y are signed int64_t  
// x0=x, x1=y, x2=result  
cmp x0, x1           // Compare x and y  
b.le .Lelse          // Invert test: jump if x <= y  
sub x2, x0, x1        // Then-body: result = x - y  
b .Lend              // Skip else block  
.Lelse:  
sub x2, x1, x0        // Else-body: result = y - x  
.Lend:
```

Assembly Pattern: Invert the high-level condition to fall through into the “then” path.

Loop Translation: Inverted Loop Pattern

High-Level C Loop

```
// Compute array sum
int64_t i = 0;
int64_t n = ...; // signed bound
while (i < n) {
    sum += arr[i];
    i++;
}
```

Bottom-Tested AArch64 Loop

```
// x0=arr, x1=n, x2=i, x3=sum
mov x2, xzr           // i = 0
mov x3, xzr           // sum = 0
b .Ltest              // Jump to test first
.Lbody:
ldr x4, [x0, x2, lsl #3] // Load arr[i]
add x3, x3, x4         // sum += arr[i]
add x2, x2, #1         // i++
.Ltest:
cmp x2, x1             // i < n?
b.lt .Lbody            // Loop if true
```

Efficiency Rule: Placing the branch at the bottom requires only one loop-closing conditional branch in the steady-state loop body.

Branch Penalties and Prediction

- Modern processors often fetch instructions before a conditional branch is resolved.
- The processor predicts the branch direction and continues along the predicted path.
- A correct prediction allows execution to continue with little disruption.
- A misprediction requires incorrect work to be discarded and execution redirected.
- The **misprediction penalty** depends on pipeline depth and front-end organization.
- Predictable branches are usually inexpensive; irregular, data-dependent branches are more difficult to predict.
- Therefore, a branch is not inherently slow: the main cost comes from **incorrect prediction**.

Branchless Conditional Selection

Conditional Select

AArch64 provides `csel` to choose between two register values using the NZCV condition flags without introducing a control-flow branch.

Signed Maximum

Suppose the desired operation is:

$$\text{result} = (x1 > x2) ? x1 : x2$$

```
cmp x1, x2           // Set NZCV from x1 - x2
csel x0, x1, x2, gt  // Signed greater-than selection
```

For unsigned values, the corresponding condition would be `hi`.

Related Branchless Helpers

When Is Branchless Code Better?

- Branchless selection avoids **branch-misprediction risk** because control flow does not depend on the condition.
- `csel` replaces a control-flow dependency with a data dependency on the condition flags.
- Branchless code is often attractive when the condition is difficult to predict or when both candidate values are already available.
- A highly predictable branch can still perform very well and may outperform conditional selection.
- Performance therefore depends on the processor microarchitecture and the behavior of the workload.

Key Point: Branchless code is not automatically faster; it trades branch-prediction risk for data dependencies.

Review and Self-Test Questions

Topic 4: Comprehensive Summary

- **Instruction Encoding:** Many AArch64 instructions encode a shifted register operand within a single instruction.
- **Bitwise Primitives:** Bitwise operations apply independently to each corresponding bit position, with native support for `and`, `orr`, `eor`, and bit clearing via `bic`; field extraction via `ubfx`.
- **Flag-Setting Forms:** Common flag-setting forms include “s” variants (`adds`, `subs`) and explicit comparisons (`cmp`, `tst`); carry flag is active ($C = 1$) when subtraction does not borrow.
- **Control Flow Idioms:** A common control-flow layout inverts the condition so one path falls through; loop inversion cuts branch overhead to one loop-closing branch in steady state.
- **Branch Elimination:** Conditional select instructions (`csel`, `cset`) avoid misprediction risk, though performance trade-offs depend on pipeline depth and dependency chains.

Self-Test Questions: Arithmetic & Logic

- ❶ **Shifted Addition:** What is the exact mathematical operation and resulting decimal value stored in `x0` after executing:

```
mov x1, #12  mov x2, #5  add x0, x1, x2, lsl #2
```

- ❷ **Bit Clear Operation:** If register `x1` contains `0xff` and `x2` contains `0x0f`, what value resides in `x0` after executing `bic x0, x1, x2`?
- ❸ **Signed vs. Unsigned Comparisons:** If `x0` contains -1 (`0xffffffff_ffffffff`) and `x1` contains `1`, which branch will be taken after `cmp x0, x1: b.lt` or `b.lo`? Why?
- ❹ **Flag Updating:** Explain why `add x0, x1, x2` does not update the Zero (Z) flag even if the resulting sum is exactly zero.

Self-Test Questions: Control Flow & Selection

- 1 **Loop Inversion Benefit:** Why do optimizing compilers place the conditional loop branch at the bottom of the loop body rather than at the top?
- 2 **Branchless Translation:** Write an equivalent 3-instruction branchless AArch64 sequence using `cmp`, `neg`, and `csel` to implement:

```
int64_t abs_val = (val < 0) ? -val : val;
```

- 3 **Fused Branch Instructions:** When should an assembly programmer use `cbz x0, label` instead of `cmp x0, #0` followed by `b.eq label`?
- 4 **Branch Misprediction Trade-off:** What factors determine whether `csel` or a conditional branch is faster?